

Task Manager — Comprendre le code React

Guide pas à pas pour un développeur venant d'Angular
qui n'a jamais écrit une ligne de React

Frontend : React 19 · TypeScript · Vite · React Router

Backend consommé : API REST Spring Boot (<http://localhost:8081>)

Code commenté en français dans tous les fichiers du dossier src/

Sommaire

1. Comment utiliser ce guide
2. L'application en un coup d'œil
3. React expliqué simplement (pour un angulariste)
4. Visite guidée du code, étape par étape
5. Les flux de l'application expliqués pas à pas
6. useState et useEffect : le cœur de React
7. Lancer, tester et déboguer
8. Mémo de traduction Angular vers React
9. Questions fréquentes de débutant
10. Et après ?

0. Comment utiliser ce guide

Ce guide suppose une seule chose : vous savez écrire une application **Angular**, et vous n'avez jamais fait de **React**. Il ne vous demande pas d'apprendre le JavaScript : tout le code est en TypeScript, que vous connaissez déjà.

Le conseil de lecture le plus important

Ouvrez le projet dans votre éditeur **à côté de ce document**. Chaque chapitre cite des fichiers réels. Tous les fichiers du dossier `src/` sont commentés en français : le guide vous donne la vue d'ensemble, les commentaires vous donnent le détail ligne par ligne.

Les trois chemins possibles

Vous voulez...	Lisez dans cet ordre
Comprendre React en général	Chapitres 2 puis 5
Comprendre CE projet	Chapitres 1, 3 puis 4
Juste lancer et tester l'application	Chapitre 6

Une différence à accepter dès maintenant : React n'est pas un framework complet comme Angular. Il n'y a ni modules, ni décorateurs, ni injection de dépendances, ni RxJS. React ne fait qu'une chose : afficher des composants en fonction de données. Tout le reste (routing, HTTP, état global) s'ajoute avec de petites librairies ou s'écrit à la main. C'est déroutant au début, puis très libérateur.

1. L'application en un coup d'œil

L'application comporte trois écrans et permet de gérer des tâches personnelles. Chaque utilisateur ne voit que ses propres tâches ; la sécurité est assurée par le token JWT envoyé à chaque requête.

Les trois écrans

URL	Écran	Fichier
<code>/login</code>	Connexion (login + mot de passe)	<code>src/pages/LoginPage.tsx</code>
<code>/register</code>	Inscription (nom, login, email, téléphone camerounais, mot de passe)	<code>src/pages/RegisterPage.tsx</code>
<code>/tasks</code>	Liste des tâches + filtres + création/modification/suppression (page protégée)	<code>src/pages/TasksPage.tsx</code>

Les fonctionnalités et leur fichier responsable

Fonctionnalité	Où c'est écrit
Appels HTTP, ajout automatique du token JWT, gestion des erreurs	<code>src/api/client.ts</code>
Connexion / inscription (appels API)	<code>src/api/auth.ts</code>
CRUD des tâches (appels API)	<code>src/api/tasks.ts</code>
Validation email et téléphone camerounais	<code>src/utils/validation.ts</code>
Libellés des statuts, formatage des dates	<code>src/utils/task.ts</code>
Message d'erreur générique	<code>src/utils/error.ts</code>
État de connexion partagé (token, login)	<code>src/context/AuthContext.ts</code> + <code>AuthProvider.tsx</code>
Logique des tâches (liste, filtre, chargement)	<code>src/hooks/useTasks.ts</code>
Affichage (navbar, cartes, modale, filtres...)	<code>src/components/</code>
Navigation entre les écrans	<code>src/App.tsx</code>

Les librairies, et leur équivalent Angular

Librairie	Rôle	Équivalent Angular
react / react-dom 19	Le cœur (composants, rendu)	@angular/core
typescript	Typage statique	TypeScript (identique)
vite	Serveur de développement + build	Angular CLI (ng serve / ng build)
react-router-dom	Navigation entre les pages	@angular/router
fetch (natif navigateur)	Requêtes HTTP	HttpClient

2. React expliqué simplement (pour un angulariste)

Ce chapitre est le plus important du guide. Une fois ces sept notions comprises, tout le code du projet devient lisible. Chaque notion est expliquée avec un mini-exemple autonome, puis reliée au projet.

2.1 Un composant est une fonction qui retourne du HTML

Oubliez la classe `@Component`. En React, un composant est une fonction JavaScript qui **retourne du HTML**. Ce HTML écrit dans du TypeScript s'appelle le **JSX** :

```
function Bonjour() {
  return <h1>Bonjour !</h1>;
}
```

On l'utilise comme une balise HTML : `<Bonjour />`. La majuscule est obligatoire — c'est ainsi que React distingue un composant d'une balise HTML réelle.

Dans le projet, `StatusBadge` est un composant très simple :

```
export function StatusBadge({ status }: StatusBadgeProps) {
  return <span className={`badge badge--${status.toLowerCase()}`}>
    {statusLabel(status)}
  </span>;
}
```

Trois différences JSX à retenir :

- On écrit `className` et non `class` (`class` est un mot réservé de JavaScript).
- Pour afficher une valeur, on met des accolades : `{status}`. En Angular : `{{ status }}`.
- Un attribut de composant est en camelCase : `onClick` (Angular : `(click)`), `value` (Angular : `[value]`).

2.2 Les props sont les paramètres de la fonction

Là où Angular écrit `@Input() nom: string`, React écrit simplement un paramètre de fonction :

```
function Bonjour({ nom }: { nom: string }) {
  return <h1>Bonjour {nom} !</h1>;
}

// Utilisation : <Bonjour nom="Awa" />
```

Le `{ nom }` entre accolades s'appelle une *déstructuration* : c'est du JavaScript moderne qui extrait la propriété `nom` de l'objet des props. Dans le projet, `TaskItem` reçoit trois props : la tâche et deux fonctions.

Communication enfant vers parent : Angular utilise `@Output()` + `EventEmitter`. React fait plus simple : le parent passe une **fonction en prop** et l'enfant l'appelle. Dans `TaskItem`, les props `onEdit` et `onDelete` sont exactement cela.

2.3 L'état local : la mémoire du composant (useState)

Un composant doit parfois se souvenir de choses : la valeur d'un champ, une liste, un booléen « en cours de chargement ». En Angular, ce sont des propriétés de classe. En React, on utilise le hook `useState`. Voici un compteur complet :

```
import { useState } from 'react';

function Compteur() {
  const [valeur, setValeur] = useState(0);

  return (
    <button onClick={() => setValeur(valeur + 1)}>
      Cliqué {valeur} fois
    </button>
  );
}
```

Décortiquons la ligne magique :

- `useState(0)` déclare un état avec la valeur initiale `0` et retourne un tableau de deux éléments.
- `const [valeur, setValeur] = ...` récupère ces deux éléments : la valeur actuelle et la fonction pour la modifier.
- `setValeur(1)` change la valeur **et demande à React de redessiner le composant**.

Règle absolue : on n'écrit **jamais** `valeur = valeur + 1`. Si on modifie la variable directement, React ne le sait pas et l'écran ne change pas. On passe **toujours** par la fonction `setValeur`.

Dans le projet, chaque champ de formulaire a son propre état. Par exemple dans `TaskFormModal` :

```
const [title, setTitle] = useState(task?.title ?? '');
const [description, setDescription] = useState(task?.description ?? '');
const [status, setStatus] = useState<TaskStatus>(task?.status ?? 'TODO');
```

Et le champ est « contrôlé » par cet état — c'est l'équivalent de `[(ngModel)]` :

```
<input
  value={title}
  onChange={(event) => setTitle(event.target.value)}
/>
```

2.4 Quand l'état change, React redessine

Voici le fonctionnement, en une phrase : **quand un état change, React ré-exécute la fonction du composant, puis compare le nouveau résultat avec l'ancien pour ne mettre à jour que ce qui a changé à l'écran.**

Conséquences pratiques :

- Le code de la fonction de composant s'exécute *à chaque rendu*. Ce n'est pas un « constructeur » : évitez-y les effets de bord (appels API, écritures).
- Il n'y a pas de « détection de changement » globale comme en Angular : React sait exactement quel composant a changé.

- Comme React compare les *références*, on ne modifie jamais un tableau ou un objet existant : on en crée un nouveau (voir chapitre 5).

2.5 useEffect : le code exécuté « après l'affichage »

Certaines choses ne peuvent pas se faire pendant le rendu : appeler le serveur, s'abonner à un événement clavier, modifier le titre de l'onglet. Pour cela, React fournit le hook `useEffect`. Exemple :

```
useEffect(() => {
  document.title = `Cliqué ${valeur} fois`;
}, [valeur]);
```

La fonction est exécutée **après** que React a affiché le composant. Le deuxième argument, le tableau `[valeur]`, s'appelle la **liste des dépendances**. Il existe trois cas :

Écriture	Quand l'effet s'exécute	Équivalent Angular
<code>useEffect(() => {...})</code>	Après chaque rendu (rarement souhaité)	—
<code>useEffect(() => {...}, [])</code>	Une seule fois, au montage	<code>ngOnInit</code>
<code>useEffect(() => {...}, [valeur])</code>	Au montage, puis à chaque fois que <code>valeur</code> change	<code>ngOnChanges</code>

Un effet peut aussi se **nettoyer** : si la fonction retourne une autre fonction, React l'appellera avant le prochain passage de l'effet et quand le composant disparaît. C'est le `ngOnDestroy` d'Angular :

```
useEffect(() => {
  const handleKeyDown = (event: KeyboardEvent) => {
    if (event.key === 'Escape') onClose();
  };

  window.addEventListener('keydown', handleKeyDown);
  return () => window.removeEventListener('keydown', handleKeyDown);
}, [onClose]);
```

Dans le projet, cet effet sert à fermer la fenêtre modale avec la touche Échap (fichier `TaskFormModal.tsx`). Le chapitre 5 approfondit `useState` et `useEffect` avec les pièges classiques.

2.6 Le contexte : partager sans injection de dépendances

React n'a pas d'injecteur. Pour que toutes les pages connaissent l'utilisateur connecté, on utilise un **contexte** : une « boîte » posée tout en haut de l'arbre des composants, que n'importe quel composant descendant peut ouvrir.

Le projet découpe cela en trois petits fichiers :

Fichier	Rôle	Analogie Angular
---------	------	------------------

<code>context/AuthContext.ts</code>	Décrit ce que la boîte contient et fournit le hook <code>useAuth()</code>	Le type du service
<code>context/AuthProvider.tsx</code>	Remplit la boîte (token, login, signIn, signOut)	Le service <code>providedIn: 'root'</code>
N'importe quel composant	Ouvre la boîte avec <code>const { login } = useAuth()</code>	<code>constructor(private auth: AuthService)</code>

Différence de fond : en Angular, l'injecteur crée *une* instance du service et vous l'injecte. En React, `AuthProvider` détient l'état, et `useAuth()` le lit. C'est presque la même chose vue de l'extérieur : `useAuth()` remplace l'injection.

2.7 Les quatre règles d'or

- Les hooks au premier niveau.** `useState`, `useEffect`, etc. s'appellent toujours au même endroit du composant, jamais dans un `if` ou une boucle. C'est pour cela que les `return <Navigate />` conditionnels (dans `LoginPage`, `RegisterPage`) sont placés *après* les hooks.
- Jamais de mutation d'état.** On remplace par une nouvelle valeur : `setTasks([nouvelle, ...tasks])`, pas `tasks.push(...)`.
- Une clé stable pour chaque élément d'une liste.** Dans un `.map()`, la prop `key` est obligatoire (équivalent du `trackBy` d'Angular).
- Les données descendent, les événements remontent.** Le parent passe des données (props) et des fonctions de rappel ; l'enfant appelle ces fonctions.

2.8 Tableau de correspondance complet

Concept	Angular	React (ce projet)
Composant	Classe avec <code>@Component</code>	Fonction qui retourne du JSX
Template	Fichier <code>.html</code> séparé	JSX dans le même fichier <code>.tsx</code>
Données entrantes	<code>@Input()</code>	Props (paramètre de fonction)
Événements sortants	<code>@Output()</code> <code>EventEmitter</code>	Fonctions passées en props
État local	Propriété de classe	<code>useState</code>
Cycle de vie	<code>ngOnInit</code> , <code>ngOnDestroy</code> ...	<code>useEffect</code> (+ nettoyage)
Services / DI	<code>@Injectable</code> + constructeur	Modules ES + <code>Context</code> + <code>useAuth()</code>
Observables	RxJS	Promesses <code>async/await</code>
Annuler une requête	<code>takeUntil(destroy\$)</code>	Drapeau <code>cancelled</code> + nettoyage d'effet
Intercepteur HTTP	<code>HttpInterceptor</code>	En-tête ajouté dans <code>client.ts</code>
Guard de route	<code>CanActivate</code>	Composant <code>ProtectedRoute</code>

Condition	<code>*ngIf</code>	<code>{condition && ...}</code> ou ternaire
Boucle	<code>*ngFor</code>	<code>.map()</code> avec <code>key</code>
Formulaire	<code>ngModel</code> / Reactive Forms	Champ contrôlé : <code>value</code> + <code>onChange</code>
Pipes	<code> date</code> , <code> uppercase</code>	Fonctions utilitaires (<code>utils/task.ts</code>)
Styles	Encapsulation par composant	CSS global (classes préfixées)
Lancer en dev	<code>ng serve</code>	<code>npm run dev</code>

3. Visite guidée du code, étape par étape

Cette visite suit exactement l'ordre conseillé dans le `README.md`. À chaque étape : le rôle du fichier, ce qu'il faut retenir, et un court extrait. Ouvrez les fichiers dans votre éditeur au fur et à mesure.

Étape 1 — `src/types/index.ts` : le vocabulaire des données

Avant de lire du code, on lit les **types** : ils décrivent tout ce qui circule entre le frontend et l'API. C'est l'équivalent de vos interfaces de modèles Angular.

```
export type TaskStatus = 'TODO' | 'IN_PROGRESS' | 'DONE';

export interface Task {
  id: number;
  title: string;
  description: string | null;
  status: TaskStatus;
  createdAt: string;
}
```

À retenir : `TaskStatus` est une union de trois chaînes : TypeScript interdit toute autre valeur. `description` peut être `null` (le backend l'autorise).

Étape 2 — `src/api/client.ts` : le seul fichier qui parle au réseau

C'est le fichier le plus important côté données. Il joue le rôle de `HttpClient` et d' `HttpInterceptor` :

- il préfixe chaque appel avec l'adresse du backend (`http://localhost:8081`) ;
- il ajoute l'en-tête `Authorization: Bearer <token>` automatiquement ;
- il transforme toutes les erreurs en `ApiError` avec le message du backend.

```
const token = getStoredToken();
if (token) {
  headers.set('Authorization', `Bearer ${token}`);
}

const response = await fetch(`${API_BASE_URL}${path}`, { ...init, headers });
```

À retenir : aucun autre fichier n'utilise `fetch`. Si l'adresse du backend change, on ne modifie qu'ici (ou via `.env`).

Étape 3 — src/api/auth.ts et tasks.ts : les services

Une fonction par endpoint. Ce sont vos services Angular, mais en version minimale :

```
export function login(credentials: Credentials): Promise<AuthenticationResponse> {
  return api.post<AuthenticationResponse>('/api/auth/login', credentials);
}

export function getTasks(status?: TaskStatus | ''): Promise<Task[]> {
  const query = status ? `?status=${status}` : '';
  return api.get<Task[]>(`/api/tasks${query}`);
}
```

À retenir : les composants ne connaissent jamais les URL de l'API ; ils appellent `login(...)`, `createTask(...)`, etc.

Étape 4 — src/utils/ : les petites fonctions pures

Fichier	Contenu
<code>validation.ts</code>	Regex email et téléphone camerounais + fonctions <code>isValidEmail()</code> , <code>isValidCameroonPhone()</code>
<code>task.ts</code>	<code>statusLabel()</code> (IN_PROGRESS → « En cours »), <code>filterLabel()</code> , <code>formatDateTime()</code> (dates en français)
<code>error.ts</code>	<code>getErrorMessage()</code> : transforme n'importe quelle erreur en message affichable

À retenir : ce sont des fonctions pures, sans état ni React. Elles remplacent les pipes Angular. Les regex sont identiques à celles du backend (`Constants.java`).

Étape 5 — src/context/AuthContext.ts : la boîte de partage

```
export interface AuthContextValue {
  login: string | null;
  isAuthenticated: boolean;
  signIn: (token: string, login: string) => void;
  signOut: () => void;
}

export const AuthContext = createContext<AuthContextValue | null>(null);

export function useAuth(): AuthContextValue {
  const context = useContext(AuthContext);
  if (context === null) throw new Error("useAuth doit être utilisé à l'intérieur d'un AuthProvider");
  return context;
}
```

À retenir : `useAuth()` est le point d'entrée unique pour connaître l'utilisateur connecté. Si un composant est utilisé hors du provider, l'erreur est immédiate et claire.

Étape 6 — src/context/AuthProvider.tsx : celui qui remplit la boîte

```
const [token, setToken] = useState<string | null>(() => getStoredToken());
const [login, setLogin] = useState<string | null>(() => getStoredLogin());

const signIn = useCallback((newToken: string, newLogin: string) => {
  storeAuth(newToken, newLogin); // 1. on écrit dans le navigateur
  setToken(newToken);           // 2. on met à jour l'état React
  setLogin(newLogin);
}, []);
```

À retenir : le formulaire `useState(() => ...)` (une fonction) ne lit le localStorage qu'une seule fois. Et c'est la mise à jour d'état (point 2) qui fait réagir toute l'application.

Étape 7 — src/hooks/useTasks.ts : la logique des tâches isolée

Un **hook personnalisé** est simplement une fonction qui commence par `use` et qui regroupe de la logique React. `useTasks` contient : la liste, le filtre, l'état de chargement, l'erreur, la fonction `changeFilter` et la fonction `refresh`.

```

export function useTasks() {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [statusFilter, setStatusFilter] = useState<TaskStatus | ''>>('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let cancelled = false;

    getTasks(statusFilter)
      .then((data) => { if (!cancelled) { setTasks(data); setError(null); } })
      .catch((loadError) => { if (!cancelled) setError(getErrorMessage(loadError)); })
      .finally(() => { if (!cancelled) setLoading(false); });

    return () => { cancelled = true; };
  }, [statusFilter]);

  // ... changeFilter et refresh ...

  return { tasks, statusFilter, loading, error, changeFilter, refresh };
}

```

À retenir : ce fichier est le seul endroit qui appelle `getTasks()` pour l'affichage. La page ne voit que des données prêtes à afficher. C'est l'équivalent d'un service Angular qui expose un état observable, mais en 60 lignes.

Étape 8 — src/components/ : les briques d'affichage

Composant	Rôle	Notion illustrée
<code>ProtectedRoute.tsx</code>	Redirige vers /login si non connecté	Guard (<code>CanActivate</code>)
<code>Navbar.tsx</code>	Barre du haut + bouton de déconnexion	<code>useAuth()</code> , <code>useNavigate()</code>
<code>StatusBadge.tsx</code>	Pastille colorée du statut	Composant sans état (props seules)
<code>TaskItem.tsx</code>	Carte d'une tâche + boutons Modifier/Supprimer	Callbacks en props (enfant → parent)
<code>TaskFilters.tsx</code>	Boutons Toutes / À faire / En cours / Terminée	Props + classe conditionnelle
<code>TasksSection.tsx</code>	Chargement, liste vide, ou liste des tâches	Retours anticipés (lisibilité)
<code>TaskFormModal.tsx</code>	Formulaire de création/modification	Formulaire contrôlé, <code>useEffect</code> , <code>async</code>

Étape 9 — src/pages/ : les trois écrans

Une page = un composant qui assemble les briques précédentes. Depuis le découpage, `TasksPage` reste courte : elle ne gère que la modale et les actions créer/modifier/supprimer.

```
const { tasks, statusFilter, loading, error, changeFilter, refresh } = useTasks();

const [modalOpen, setModalOpen] = useState(false);
const [editingTask, setEditingTask] = useState<Task | null>(null);
```

À retenir : les pages font le lien entre la logique (hooks, api) et l'affichage (composants).

Étape 10 — src/App.tsx : le plan de navigation

```
<Routes>
  <Route path="/login" element={<LoginPage />} />
  <Route path="/register" element={<RegisterPage />} />
  <Route path="/tasks" element={
    <ProtectedRoute><TasksPage /></ProtectedRoute>
  } />
  <Route path="*" element={<Navigate to="/tasks" replace />} />
</Routes>
```

À retenir : lire ce fichier de haut en bas donne la carte complète de l'application.

Étape 11 — src/main.tsx : le démarrage

```
createRoot(document.getElementById('root')).render(
  <StrictMode><App /></StrictMode>,
);
```

À retenir : `StrictMode` double certains traitements *en développement uniquement* pour détecter les effets mal nettoyés. C'est pour cela que `useTasks` utilise le drapeau `cancelled`.

4. Les flux de l'application expliqués pas à pas

4.1 Au démarrage de l'application

```
index.html (contient <div id="root">)
└─ main.tsx      createRoot(...).render(<App />)
    └─ App.tsx    BrowserRouter > AuthProvider > Routes
        ├── URL /login    → LoginPage
        ├── URL /register → RegisterPage
        └─ URL /tasks     → ProtectedRoute → TasksPage
```

Au premier affichage, `AuthProvider` lit le `localStorage` : si un token y est présent, l'utilisateur est considéré connecté immédiatement.

4.2 Inscription (bouton « S'inscrire »)

```
RegisterPage
├ 1. validate()                vérifie nom, login, email, téléphone, mot de passe
├ 2. register(payload)        POST /api/auth/register
│   └ client.request()        ajoute JSON + appelle fetch
│       └ backend Spring      valide, hache le mot de passe, crée l'utilisateur
│           └ réponse          { accessToken, message }
├ 3. signIn(accessToken, login) localStorage + état du contexte
└ 4. navigate('/tasks')       React Router affiche TasksPage
```

En cas de refus du backend (login déjà utilisé, email invalide...), l'exception `ApiError` remonte et le tableau `errors` s'affiche en rouge au-dessus du formulaire.

4.3 Connexion

```
LoginPage
├ 1. contrôle local           champs non vides
├ 2. login({ login, password }) POST /api/auth/login
│   └ backend                 compare le mot de passe (BCrypt) + compte actif
│                               puis génère un JWT valable 24 h
└ 3. signIn + navigate('/tasks')
```

4.4 Afficher les tâches (le chemin le plus important)

```
TasksPage
└ useTasks()
    └ useState(...)           crée tasks / statusFilter / loading / error
        └ useEffect sur [statusFilter]
            └ getTasks(statusFilter)    src/api/tasks.ts
                └ api.get<Task[]>(...)  src/api/client.ts
                    └ fetch('/api/tasks?status=...',
                        { Authorization: 'Bearer eyJ...' })
                        └ backend Spring : JwtFilter identifie l'utilisateur
                            └ réponse : uniquement SES tâches
```

Deux subtilités expliquées dans les commentaires du hook :

- Quand le filtre change, l'effet se relance (l'ancien résultat est ignoré grâce à `cancelled`).
- En développement, l'appel part deux fois (StrictMode) : c'est normal, seul un résultat est conservé.

4.5 Créer ou modifier une tâche

```
Clic « Nouvelle tâche »      → openCreateModal() → editingTask = null, modalOpen = true
Clic « Modifier » d'une tâche → openEditModal(task) → editingTask = task, modalOpen = true
```

```
TaskFormModal (saisie)
└─ submit → handleSubmit(payload)      (dans TasksPage)
    │ si editingTask → updateTask(id, payload)  PUT /api/tasks/{id}
    │ sinon          → createTask(payload)      POST /api/tasks
    │ closeModal()
    └─ refresh()                                recharge la liste depuis le serveur
```

Si le serveur refuse (titre vide, tâche d'un autre utilisateur...), `handleSubmit` laisse l'erreur remonter : la modale l'attrape et l'affiche sans se fermer.

4.6 Supprimer une tâche

```
Clic « Supprimer »
└─ window.confirm('Supprimer la tâche « ... » ?') → si Annuler, on s'arrête
└─ removeTask(id)                                DELETE /api/tasks/{id}
└─ refresh()                                      recharge la liste
```

4.7 Déconnexion et protection des pages

```
Navbar → Se déconnecter
└─ signOut()      efface le localStorage + vide le contexte
└─ navigate('/login')
```

Ensuite, toute tentative d'aller sur /tasks :

```
ProtectedRoute → isAuthenticated = false → <Navigate to="/login" />
```

4.8 Résumé : qui appelle quoi

Qui	Appelle	Pour quoi faire
Une page	Un hook (<code>useTasks</code>) ou un contexte (<code>useAuth</code>)	Obtenir des données / des actions
Un hook	Un service api (<code>getTasks</code> ...)	Déclencher une requête
Un service api	<code>client.request</code>	Ajouter l'URL, le token, gérer les erreurs
<code>client.request</code>	<code>fetch</code>	Parler au backend Spring
Une page	Des composants d'affichage	Rendre le résultat

5. useState et useEffect : le cœur de React

Ce chapitre approfondit les deux hooks que vous utiliserez 90 % du temps, avec les questions que se pose tout débutant.

5.1 Pourquoi `const [valeur, setValeur] = useState(0)` ?

`useState` retourne un tableau de deux éléments qu'on « déballe » en une ligne :

- l'élément 0 est la valeur actuelle ;
- l'élément 1 est une fonction pour changer la valeur (et prévenir React).

On aurait pu écrire, en plus long :

```
const paire = useState(0);
const valeur = paire[0];
const setValeur = paire[1];
```

5.2 Pourquoi la valeur ne change-t-elle pas tout de suite ?

Les mises à jour d'état sont « asynchrones » : React attend la fin du gestionnaire d'événement pour redessiner. Donc ceci est trompeur :

```
setCount(count + 1);
console.log(count); // affiche encore l'ANCIENNE valeur
```

Si vous devez utiliser la valeur précédente, passez une **fonction** à la mise à jour :

```
setCount((precedent) => precedent + 1);
```

Règle pratique : dès que la nouvelle valeur dépend de l'ancienne, utilisez la forme `setX((precedent) => ...)`.

5.3 Pourquoi ne jamais modifier un tableau ou un objet de l'état ?

React compare les références pour savoir si quelque chose a changé. Si vous modifiez le tableau existant, la référence reste la même et React peut manquer la mise à jour.

```
// Interdit
tasks.push(nouvelleTache);
setTasks(tasks);

// Correct : un NOUVEAU tableau
setTasks([...tasks, nouvelleTache]);

// Correct : remplacer un élément par son id
setTasks(tasks.map((t) => (t.id === id ? { ...t, title: 'Nouveau titre' } : t)));
```

Le `{ ...t, title: ... }` s'appelle un *spread* : il copie l'objet puis remplace une propriété. C'est le réflexe à acquérir.

5.4 Pourquoi mon useEffect se déclenche-t-il deux fois ?

Parce que vous êtes en mode développement et que `<StrictMode>` exécute volontairement chaque effet deux fois (montage, démontage, remontage) pour vérifier que le nettoyage est correct. En production : un seul appel. Dans ce projet, le drapeau `cancelled` fait que seul le dernier résultat est conservé.

5.5 Comment éviter les boucles infinies dans useEffect ?

```
// Boucle infinie : l'effet modifie ce qu'il observe
useEffect(() => {
  setTasks([...tasks, x]);
}, [tasks]);

// Correct : l'effet ne dépend que de ce qui doit le relancer
useEffect(() => {
  getTasks(statusFilter).then(setTasks);
}, [statusFilter]);
```

Question à se poser : « qu'est-ce qui doit relancer cet effet ? » La réponse donne la liste des dépendances. Si la liste change trop souvent, ne mettez pas d'objet ou de tableau créé à chaque rendu dedans.

5.6 Le nettoyage d'effet, à quoi ça sert vraiment ?

Trois cas typiques dans ce projet :

Effet	Nettoyage	Pourquoi
Écouter la touche Échap (modale)	<code>removeEventListener</code>	Sinon l'écouteur reste en mémoire après fermeture
Charger les tâches	<code>cancelled = true</code>	Ignorer une réponse si le filtre a changé entre-temps
S'abonner à un flux (non utilisé ici)	se désabonner	C'est exactement le <code>unsubscribe</code> que vous mettez dans <code>ngOnDestroy</code>

5.7 Et un hook personnalisé, c'est quoi ?

Juste une fonction dont le nom commence par `use` et qui appelle d'autres hooks. `useTasks` en est un : il cache quatre `useState`, un `useEffect` et deux fonctions, et expose seulement ce qui est utile. C'est l'équivalent d'un service Angular spécialisé, mais il vit et meurt avec le composant qui l'utilise.

5.8 Les trois erreurs de débutant les plus fréquentes

1. **Appeler un hook dans un `if`** → erreur « Rendered fewer hooks than expected ». La solution : sortir le `if` du composant ou le placer après tous les hooks.

2. **Oublier la prop** `key` dans un `.map()` → avertissement dans la console et comportement bizarre à la suppression. Utilisez toujours `key={element.id}`.
3. **Mettre** `value` **sans** `onChange` dans un champ → le champ devient impossible à modifier. Un champ contrôlé a toujours les deux.

6. Lancer, tester et déboguer

6.1 Lancer le backend

```
cd task_manager_backend
.\mvnw.cmd spring-boot:run
```

MySQL doit être démarré. L'API écoute sur `http://localhost:8081`. Un compte existe déjà : `admin` / `12345`.
Swagger : `http://localhost:8081/swagger-ui.html`.

6.2 Lancer le frontend

```
cd task_manager_frontend
npm install      # une seule fois
npm run dev      # http://localhost:5173
```

6.3 Vérifier que le code est sain

```
npm run build    # compilation TypeScript + build de production
npm run lint     # règles de qualité (dont les règles des hooks)
```

6.4 Déboguer dans le navigateur (F12)

Onglet	À quoi il sert ici
Console	Lire les erreurs JavaScript et les avertissements React (ex. « missing key »)
Network / Réseau	Voir les appels <code>/api/tasks</code> : statut (200, 400, 403), en-tête <code>Authorization</code> , corps de la réponse
Application	Vérifier le token : Application → Local Storage → <code>tm_token</code>

6.5 Les erreurs fréquentes et leur solution

Message	Cause probable	Solution
« Impossible de contacter le serveur. Vérifiez que le backend est démarré. »	Le backend ne tourne pas	Lancer Spring Boot (6.1)

« Votre session a expiré, veuillez vous reconnecter »	Token expiré (24 h) ou invalide	Se reconnecter
403 Forbidden sur <code>/api/tasks</code>	Aucun token envoyé	Vérifier <code>tm_token</code> dans le localStorage, puis se reconnecter
« Ce login est déjà utilisé »	Normal : validation du backend	Choisir un autre login
Le champ de saisie ne réagit pas	<code>value</code> sans <code>onChange</code>	Ajouter <code>onChange={(e) => setX(e.target.value)}</code>

7. Mémo express Angular vers React

Les 12 réflexes à garder sous la main :

Vous pensez...	Vous écrivez...
Composant + template	Une fonction qui retourne du JSX
<code>@Input()</code>	Paramètre de fonction typé
<code>@Output()</code>	Une prop de type <code>(valeur) => void</code>
Propriété de classe	<code>useState</code>
<code>ngOnInit</code>	<code>useEffect(() => {...}, [])</code>
<code>ngOnDestroy</code>	La fonction de nettoyage retournée par l'effet
<code>ngOnChanges</code>	<code>useEffect(() => {...}, [valeur])</code>
Service injecté	Fonctions importées ou <code>useContexte()</code>
<code>*ngIf</code>	<code>{condition && ...}</code>
<code>*ngFor</code>	<code>{liste.map((x) => <El key={x.id} />)}</code>
<code>[(ngModel)]</code>	<code>value={x} onChange={(e) => setX(e.target.value)}</code>
<code> date</code>	<code>formatDateTime(iso)</code>

8. Questions fréquentes de débutant

Où est le `ngOnInit` ?

C'est `useEffect(() => { ... }, [])`. Le tableau vide signifie « une seule fois au montage ».

Où sont les services et l'injection de dépendances ?

React n'en a pas. Les « services » sont des fichiers de fonctions importées (`src/api/`), et le partage global passe par un contexte (`src/context/`).

Où est le fichier HTML du composant ?

Il n'existe pas : le HTML est écrit dans la fonction, en JSX, juste avant le `return`.

Pourquoi des parenthèses après `return` ?

Pour retourner du JSX sur plusieurs lignes sans ambiguïté : `return ( <div>...</div> );`. C'est une simple convention d'écriture.

Pourquoi `className` et pas `class` ?

Parce que JSX est du JavaScript, et que `class` y est un mot réservé. React a choisi `className`.

Pourquoi mon appel API part-il deux fois au démarrage ?

C'est `<StrictMode>` en développement, pour vérifier que le nettoyage des effets est correct. En production, un seul appel. Le drapeau `cancelled` de `useTasks` garantit qu'un seul résultat est utilisé.

Comment ajouter une nouvelle page (par exemple `/profile`) ?

- 1) Créer `src/pages/ProfilePage.tsx` avec une fonction `ProfilePage` qui retourne du JSX.
- 2) L'importer dans `src/App.tsx`.
- 3) Ajouter une ligne : `<Route path="/profile" element={<ProfilePage />} />` (l'entourer de `<ProtectedRoute>` si la page doit être réservée aux connectés).
- 4) Ajouter un `<Link to="/profile">` là où vous voulez le lien.

Comment ajouter un champ à un formulaire ?

Trois ajouts : un `useState` pour la valeur, un bloc `<div className="field">` avec un `<input value={...} onChange={...} />`, et l'ajout du champ dans l'objet envoyé au serveur. Le CSS `.field` s'occupe de la mise en forme.

Puis-je utiliser RxJS ?

Oui techniquement, mais ce n'est pas nécessaire ici : les appels HTTP sont des promesses `async/await`, plus simples à lire pour ce type d'application. Les Observables restent utiles pour des flux d'événements complexes.

9. Et après ?

Exercices conseillés sur ce projet (du plus simple au plus complet)

1. **Ajouter un champ « priorité »** à la tâche (types, formulaire, affichage) : c'est l'exercice complet minimum pour pratiquer tout le cycle.
2. **Ajouter une recherche par titre** au-dessus des filtres, avec un `useState` supplémentaire.
3. **Trier par titre ou par date** : ajouter un bouton de tri (le backend accepte déjà des critères de tri).
4. **Déconnexion automatique** quand le token expire : dans `client.ts`, réagir aux réponses 401/403.
5. **Mode sombre** : des variables CSS suffisent (voir `index.css`).

Ressources recommandées

- **fr.react.dev** — la documentation officielle en français, excellente et pédagogique. Le chapitre « Penser en React » est la suite logique de ce guide.
- **reactrouter.com** — la documentation du routeur.
- **developer.mozilla.org** — pour `fetch` et les promesses.

Le meilleur exercice reste ce projet lui-même : chaque fichier de `src/` est commenté en français. Quand une ligne vous résiste, cherchez le commentaire au-dessus : il explique le « pourquoi », pas seulement le « quoi ».

Document généré pour le projet Task Manager. Édition 2 : reflète l'organisation actuelle du code (hook `useTasks`, composants `TaskFilters` et `TasksSection`, `utils/error.ts`, renommage `loginValue`) et tous les fichiers commentés. Vérifié par `npm run build` et `npm run lint` sans erreur.